2771N

Continuous Pursuit-Evasion with Aerodynamic Constraints

Computer Science Tripos – Part II

April 29, 2026

Declaration of Originality

I, the candidate for Part II of the Computer Science Tripos with Blind Grading Number **2771N**, hereby declare that this report and the work described in it are my own work, unaided except as may be specified below, and that the report does not contain material that has already been used to any substantial extent for a comparable purpose. In preparation of this report, I adhered to the Department of Computer Science and Technology AI Policy. I am content for my report to be made available to the students and staff of the University.

Date: April 29, 2026

Proforma

Candidate number:	2771N
Project Title:	**Continuous Pursuit-Evasion with Aerodynamic Constraints**
Examination:	Computer Science Tripos – Part II, 2026
Word Count:	WORD COUNT[1]
Code Line Count:	LINE COUNT[2]
Project Originator:	The candidate
Day-To-Day Supervisor:	Ramsey Faragher
Marking Supervisor:	Alastair Beresford

Original Aims of the Project

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum. Fusce ac turpis quis ligula lacinia aliquet. Mauris ipsum. Nulla metus metus, ullamcorper vel, tincidunt sed, euismod in, nibh.

Work Completed

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum. Fusce ac turpis quis ligula lacinia aliquet. Mauris ipsum. Nulla metus metus, ullamcorper vel, tincidunt sed, euismod in, nibh.

Special Difficulties

None.

[1] This word count was computed using `texcount`.
[2] This code line count was computed using `tokei` (excluding autogenerated test output).

Contents

1 Introduction **1**

1.1 Motivation 1

1.2 Project Aims 1

1.3 Success Criteria 2

2 Preparation **3**

2.1 Related Work 3

2.2 Starting Point 4

2.3 Theory and Design 4

2.3.1 Markov Decision Processes 4

2.3.2 `FastMDP` Algorithm 5

2.3.3 Reward Functions 5

2.3.4 Kinematics 6

2.3.5 Capturing 7

2.3.6 Codebase Architecture 7

2.4 Requirements Analysis 8

2.5 Development Methodology 9

3 Implementation **11**

3.1 Repository Overview 11

3.2 Kinematic Model 12

3.2.1 One- and Two-Dimensionality 13

3.2.2 Three-Dimensionality 14

3.2.3 Capturing 16

3.3 Decision Making 17

3.3.1 `FastMDP` 17

3.3.2 Parameter Analysis . . . 19
3.3.3 Reward Functions . . . 20
3.4 Output System . . . 22
3.4.1 Plots and Videos . . . 23
3.4.2 Terminal Dashboard . . . 23
3.4.3 Interactive Visualisation . . . 24
3.5 Extensions . . . 26
3.5.1 Numba Compilation . . . 26
3.5.2 Multi-Agent Runs . . . 27
3.5.3 Alternative Behaviours . . . 27

4 Evaluation **28**
4.1 Functional Evaluation . . . 28
4.1.1 Validity Definitions . . . 29
4.1.2 Observations . . . 29
4.1.3 Results Summary . . . 31
4.2 Engineering Performance . . . 32
4.3 Visualisation Evaluation . . . 32
4.4 Extension Validity . . . 34
4.5 Software Methodology Evaluation . . . 34
4.5.1 Testing Strategy . . . 34
4.5.2 Agile Effectiveness . . . 35

5 Conclusions **36**
5.1 Summary of Work . . . 36
5.2 Lessons Learnt . . . 36
5.3 Contributions to the Field . . . 37
5.4 Future Work . . . 37

Bibliography **38**

A Sprint Reviews **39**

B Configuration Values **40**

C Validation Scenarios **42**

D Proposal **43**

Chapter 1

Introduction

1.1 Motivation

Pursuit-evasion problems model scenarios where agents compete to intercept or evade one another. The problem has been studied across a wide range of scales and environments, from small discrete spaces such as chessboards to high-dimensional unbounded continuous arenas. Common variations include considering all agents as both pursuers and evaders, varying kinematic ratios between agents, and investigating teamwork or collaborative strategies [1].

The problem is also well-studied problem in aerospace systems research, with practical applications in autonomous air combat, unmanned aerial vehicle (UAV) swarm coordination, and airspace collision avoidance. Approaching real-time solutions to these problems is challenging: the state space of even a simple scenario is continuous and high-dimensional. Exact solutions are computationally intractable[2], but Markov Decision Processes provide a performant framework for efficient approximation[3], and have been shown to scale to multi-agent teams in continuous 3D state spaces[4].

This project implements and extends the FastMDP algorithm of Bertram et al.[3] in a 3D kinematic simulation of fixed-wing aircraft. The primary focus is pursuit-evasion behaviour, investigated through novel reward functions and performance analysis. A key contribution is demonstrating that the same simulation and decision-making framework generalises to other problem spaces entirely, including collaborative flight and search-and-rescue, with changes only to the reward structure.

The project is implemented in Python 3.13.2, using `numpy` and `numba` for numerical computation.

1.2 Project Aims

The core aims of this project are to:

1. Implement a kinematically accurate 3D simulation of fixed-wing aircraft using a pseudo-6DoF model.
2. Implement the FastMDP algorithm to drive behaviours between competing agents.
3. Design reward functions that produce valid pursuit-evasion behaviour.
4. Analyse the engineering performance of the simulation and decision-making framework.

After completing the core project, the following extensions were investigated:

5. Scale the simulation from 1v1 encounters to multi-agent teams.
6. Demonstrate that the decision-making framework is applicable to alternative problem spaces, requiring changes only to the reward structure.

1.3 Success Criteria

The following criteria were used to evaluate the success of the project:

1. **Behavioural validity**. The implemented FastMDP algorithm produces valid pursuit-evasion behaviour according to some definable criteria. This is the primary functional goal of the project and directly validates the correctness of the decision-making and reward framework.

2. **Kinematic validity**. Agent kinematics remain within realistic bounds across all simulation runs, including velocity, turn rate and altitude constraints. This validates the correctness of the kinematic model independently of the decision-making framework.

3. **Computational performance**. Agent actions are computed within a timeframe suitable for real-time simulation with appropriate optimisations investigated and evaluated.

4. **Extension validity**. Multi-agent teams and alternative problem spaces produce valid and consistent behaviour under the same framework.

The remainder of the dissertation covers the preparation, implementation and evaluation carried out to meet these criteria.

Chapter 2

Preparation

2.1 Related Work

Pursuit-evasion problems were first formalised through a framework for reasoning about intelligent adversaries by Isaacs[5] and remain an active area of research. Well-known examples include the discrete two-dimensional 'wolf-and-sheep' puzzle, where a wolf agent attempts to corner a sheep agent on a chessboard, or the continuous 'lion-and-man' problem which asks whether a pursuer can always catch an identical evader in a bounded arena. The expansion into an aerospace environment introduces additional kinematic modelling complexity and increases the available parameters that drive pursuit or evasion behaviour.

Determining optimal strategies in real-time or close-to real time is computationally challenging as the problem grows in dimensionality — the state-action space becomes too large to represent with traditional methods[6, 7]. This 'curse of dimensionality'[8] motivates a variety of approximation approaches. McGrew et al.[2] demonstrated an approximate dynamic programming (ADP) approach to 1v1 air combat, involving a number of additional reward-shaping techniques. While effective, this approach is limited to 2D level flight at fixed velocity, does not approach the infinite state space of continuous flight[4], and did not scale to multi-agent teams. These limitations motivate a different approach for multi-agent pursuit-evasion in a continuous 3D environment.

Bertram et al.[3] proposed a novel algorithm to closely approximate the solution to a class of MDPs with sparse rewards which they term FastMDP. Instead of well-known value iteration or policy iteration methods[8, 9], FastMDP reframes the MDP as a graph and computes the value function from individual reward sources, combining positive and negative peaks into a single surface. The algorithm has no dependency on the size of the state space, making it suitable for continuous spaces and real-time computation. This was initially demonstrated in a 2D UAV path planning context, showing scaling with the number of obstacles.

Further works by Bertram and Wei[4] expanded FastMDP to a fully continuous 3D state space, incorporating a pseudo-6DoF kinematic model of fixed-wing aircraft and driving pursuit-evasion behaviours. That work forms the basis for the simulation and decision-making framework developed in this project.

[REVIEW: NECESSARY?] Paragraph/justification about moving forwards with the FastMDP influence in my implementation, but what were my differentiations? Primarily reward structure, performance analysis alternative behaviour extensions. Also could describe how my work specifialcly involves agents that are both pursuing and evading at the same time, instead of other examples like 1 pursuer and 1 evader. Don't wnat to end up just repeating the introduction Motivation section basically

2.2 Starting Point

I had experience with building and applying Markov models in Python (in particular Markov Chains and Hidden Markov Models) from my Part IA Machine Learning and Real-World Data course, but I had not specifically implemented a Markov Decision Process or the FastMDP algorithm before.

I did not have experience with pursuit-evasion problems or explicit kinematic models, but I had written Java code to simulate physical three-dimensional systems before with a focus on linear algebra and vector mathematics rather than aerodynamics.

I was confident with the Python programming language and the tooling around it including virtual environments, the pip package manager and the pytest testing framework. I had real-world corporate experience in managing large Python projects, and was also comfortable working with the Git version control system and GitHub Actions for development workflows.

The final project is not based on existing codebases or machine learning libraries, but I examined the algorithms and pseudocode of the works described in Section 2.1 to influence my implementations.

2.3 Theory and Design

I prepared for implementation by researching and understanding two substantial bodies of theory: Markov Decision Processes and their approximation solution methods, and models of 3D kinematics.

2.3.1 Markov Decision Processes

A Markov Decision Process (MDP) is a mathematical framework for sequential decision-making under uncertainty. An MDP is a tuple (S, A, T, R) of states S, actions A, transition probability function $T : S \times A \times S \to [0, 1]$, and reward function $R : S \times A \to \mathbb{R}$ that encodes desired behaviour. A policy is a mapping $\pi : S \to A$ with a value function $V^\pi : S \to \mathbb{R}$ encoding the expected reward from following that policy (with some discount factor γ per transition), and the optimal policy π^* is the one that maximises V^{π^*} for all $s \in S$. Solving an MDP typically involves finding an optimal policy that covers all states.

A Bellman optimality equation can exactly represent the optimal value function[8].

$$V^{\pi^*}(s) = \max_{a \in A} \left[R(s, a) + \gamma \sum_{s' \in S} T(s, a, s') V^{\pi^*}(s') \right] \tag{2.1}$$

Intuitively, the best value of an optimal-policy state must equal the expected return of the best action[9]. A standard method to determine the exact solution to an MDP is value iteration, which iteratively applies the optimality equation until convergence, but as discussed becomes intractable for large state spaces and inapplicable for infinite ones. A key observation for some

approximation approaches is to identify a class of MDPs with *sparse rewards*. In a sparse-reward MDP, most states have reward $R(s, a) = 0$, and there may be reward peaks and wells localised around certain regions of states. In this situation, the transition function T and full reward function R do not need explicit construction — the value function can be approximated directly from the reward sources, bypassing standard iteration techniques.

2.3.2 `FastMDP` Algorithm

The FastMDP algorithm exploits sparse rewards by constructing the value function from positive peaks and negative wells in a continuous state space. A measure of distance $\delta : S \times S \to \mathbb{R}$ must be defined for the state space to interpret the effect of each peak. For a continuous 3D environment this can be Euclidean distance.

Each positive reward places an exponentially decaying peak, and the positive value function is defined as the max over all peaks for some state s:

$$V^+(s) = \max_{i \in R^+} \left[r_i \cdot \gamma_i^{\delta(s, s_i)} \right] \tag{2.2}$$

where R^+ are the positive peaks, r_i is the reward magnitude, $\gamma_i \in (0, 1)$ is the per-source decay factor, and $\delta(s, s_i)$ is the distance measure. The negative value function is defined similarly, but negative rewards ("risk wells") are first converted into *standard positive form* in order to calculate the value function identically to positive rewards before negating again.

$$V^-(s) = -\max_{j \in R^-} \left[|r_j| \cdot \gamma_j^{\delta(s, s_j)} \right] \tag{2.3}$$

Summing the positive and negative value functions gives a close approximation of solving the MDP with a full reward function[4, 3]. `FastMDP` also requires *forward projection* functionality, where a state s and action a are used to compute a resulting state. As the kinematic model provides deterministic transitions $s'_a = f(s, a)$, forward projection replaces the stochastic transition function T entirely. An approximation of the optimal policy can be retrieved from the value function and forward projection, and applied to determine optimal actions for any particular state.

$$V(s) = V^+(s) + V^-(s) \approx V^{\pi^*} \tag{2.4}$$

$$\pi(s) = \arg\max_{a \in A} V(s'_a) \approx \pi^* \tag{2.5}$$

2.3.3 Reward Functions

`FastMDP` has no reliance on the size of the state space, making it well-suited to the continuous 3D environment of pursuit-evasion. At each timestep, each agent formulates its own MDP with reward sources based on its opponents and their kinematic properties. It is the parameters of these reward sources (position, magnitude, decay) that drive the agent behaviour to capture enemies and avoid being captured.

One of the project success criteria was to design novel reward functions that drive this behaviour. Bertram et al. place rewards solely based on agent position — opponents have positive peaks at their current positions and negative wells at projected positions, attracting pursuers and repelling collisions. I wanted to expand on this by considering velocity, heading and alignment parameters to inform reward sources. Two agents at equal distances from a target would have different tactical advantages depending on their alignment towards the target, and if their opponent is relatively aligned towards them.

This motivates incorporating *velocity direction* into the reward structure. In addition to positions, my MDP formulation would provide each agent with access to additional state for all opponents:

1. The unit displacement vector $\hat{r}_{pe}$ from self to enemy.
2. The self velocity vector v_{self}.
3. The enemy velocity vector v_{enemy}.
4. The scalar distance d.

Alignment measures and speed values can be derived from these quantities and used to inform the rewards. The specific reward functions built from these quantities and the parameter analysis used to refine them are described in the Implementation chapter.

2.3.4 Kinematics

The kinematic model follows the work of previous implementations[4, 10, 11]. A pseudo 6-degrees-of-freedom model (pseudo-6DoF) was applied, where an aircraft state has three positional (x, y, z) coordinates and three rotational (roll, pitch, yaw) coordinates. A full 6DoF model accounts for all aerodynamic forces acting on a fixed-wing aircraft, but the pseudo-6DoF model simplifies this to a smaller set of control inputs and approximated forces for use in the equations of motion.

An aircraft is modelled by a position (x, y, z) with altitude $h = z$, azimuth heading angle ψ, attack angle α, roll angle ϕ, flight path angle γ, and airspeed V. Position is measured in metres, speed in metres/second and all angular quantities in radians. The control inputs to an aircraft are the acceleration derived from thrust n_x, attack angle rate of change $\dot{\alpha}$, and roll angle rate of change $\dot{\phi}$. These inputs provide a reasonable approximation of throttle and joystick controls in a highly manoeuvrable aircraft.

The equations of motion are implemented directly from Bertram and Wei[4] with a few additions. Available normal force n_f is modelled as speed-dependent, reflecting that a fixed-wing aircraft cannot sustain high G-loads at low airspeed, with induced drag approximated as proportional to n_f^2 to reduce speed during hard turns. Kinematic bounds were applied per-agent rather than per-team, and a numerical guard prevents division-by-zero in the azimuth angle calculation.

$$n_f = \alpha \cdot \min\left(G_{max},\ G_{max} \cdot \left(\frac{V}{V_c}\right)^2\right) \tag{2.6}$$

$$\dot{V} = g[n_x - k_d n_f^2 - \sin\gamma] \tag{2.7}$$

$$\dot{\gamma} = \frac{g}{V}[n_f \cos\phi - \cos\gamma] \tag{2.8}$$

$$\dot{\psi} = \frac{g \cdot n_f \sin \phi}{V \cos \gamma} \tag{2.9}$$

where G_{max} is the structural G limit, V_c is the corner velocity, k_d is a drag coefficient, and $g = 9.80665$ m/s^2. The bounds were determined during implementation and enforced through clipping functions. A hard deck was implemented at $z = 0$, but this was enforced as a decision penalty rather than a kinematic limit.

A pseudo-6DoF model was appropriate for the project as the aim was to examine pursuit-evasion decision behaviour with a kinematic model of reasonable fidelity. The aerodynamic force calculations of a full-6DoF that map surface deflections to drag, lift and lateral forces were not necessary.

2.3.5 Capturing

A capturing mechanic was required to define when one agent successfully intercepts another agent without being intercepted first. Without a well-defined condition, the pursuit-evasion problem has no clear resolution — agents could approach indefinitely without a meaningful outcome.

The initial design followed Bertram and Wei[4], and maintained a *control point* for each agent that was the position the agent was in some number of timesteps ago. Capture was triggered when a pursuing agent came within a certain radius of this control point whilst sufficiently aligned towards the evader, and held this condition for a configurable number of timesteps. The intuition is that a pursuer positioned closely behind an evader and pointing towards them is in a tactically dominant position — analogous to a 'weapons lock' in air combat. This formulation was straightforward to implement and directly comparable to the existing work, and the one I was prepared to implement before beginning the project.

However, this design did not account for the relative orientations of the two agents beyond the pursuer's alignment. For example, two agents at equal distances and headings towards a target would be treated identically regardless of whether the target was aligned back towards them — a tactically significant distinction. A better capture condition would incorporate this asymmetry, ensuring that capture was only possible when an agent had a meaningful advantage over its opponent.

During implementation the initial design was found to not produce realistic capture behaviour when examined qualitatively in the visualisation, prompting the redesign of the criteria to include further alignment checks. The final capturing conditions and their implementation are described in the Implementation chapter.

2.3.6 Codebase Architecture

I laid out a high-level architecture for the codebase that matched with the MDP and kinematic frameworks investigated above. This had a clear separation of concerns: the simulation could be run independently; the MDP constructor took only the initial parameters required to solve it; and outputs could easily be generated from simulation state at each timestep. The initial design was object-oriented — a central manager constructed simulation instances that maintained lists

of agents. Each agent constructed a single MDP for the entire simulation run that they updated with new values at each timestep.

[REVIEW: figure here? i think I have some really old sketches on my ipad but they're very sloppy/incomplete? i could try and clean them up and try to make them actually demonstrate some of the original planned architecture but they would very much be filled-in (and i dont remember a lot of the original...)]

Early on in development this changed so that agents were constructing new MDP instances at each timestep, as there was minimal performance overhead and this better represented the dataflow of the program. The Implementation chapter further describes how the design eventually moved away from pure OOP in order to take advantage of various performance optimisations that did not interface well with Python objects.

[REVIEW: possibly more detail needed here? i didn't lean in too hard to planning out every piece of architecture as it was pretty straightforward - 3 modules connected, initially OOP design patterns as described. is there anything else I could/should add?]

Python had a broad ecosystem that supported all areas of the project. NumPy provided efficient numerical compute, and a variety of libraries helped with static, dynamic, and interactive visualisations. Beyond library availability, its dynamic typing and interpreted nature allowed for more rapid prototyping whilst exploiting Numba's JIT functionality to ensure performance-heavy functions could be compiled to optimised machine code.

2.4 Requirements Analysis

Before beginning the project, I performed a requirements analysis using the *MoSCoW* system. The *Must* requirements map directly onto the (non-extension) success criteria defined in the Introduction chapter.

Requirement	Priority
3D kinematic simulation of fixed-wing aircraft using a pseudo-6DoF model	`Must`
FastMDP-based decision making framework for pursuit-evasion behaviour	`Must`
Agent actions are computed within a timeframe suitable for real-time simulation	`Must`
Valid kinematic and behavioural outputs verified against defined criteria	`Must`
Output system producing video or visualisation results of simulation runs	`Should`
Reward functions are configurable without changes to core implementation	`Should`
Codebase is modular and extensible to alternative problem spaces	`Should`
Scale to multi-agent teams beyond 1v1	`Could`
JIT compilation via `numba` for performance gains	`Could`
Demonstrate framework applicability to alternative problem spaces	`Could`
Full aerodynamic modelling with surface deflections and force calculations	`Won't`
Real-world hardware deployment	`Won't`

Table 2.1: MoSCoW requirements analysis

2.5 Development Methodology

The project was developed on a personal MacBook Pro (2021, Apple M1 Pro, 16GB RAM) running Python `v3.13.2`. Key libraries were verified for compatibility before implementation with particular attention to `numba`, which supports ARM-based Apple Silicon[12] but required testing to confirm JIT compilation behaved correctly.

Git was set up for version control throughout development, with source code, assets and dissertation drafts regularly committed to a remote GitHub repository.

An agile approach to software development was adopted throughout the project. The milestones from the project proposal were used as starting points for implementation sprints, with a short written review after each 2-3 week sprint. These reviews are included in Appendix A. This approach allowed flexibility throughout the academic year as other in-term commitments and vacations affected the development schedule, which is also reflected in the commit frequency of the repository.

Unit tests were generated for individual modules using the `pytest` framework and produced alongside each implementation. A GitHub Actions workflow was configured to run the full test suite on every remote commit, with results and coverage reports saved as build artefacts. This provided a record of test outcomes throughout development to be referenced in the Evaluation chapter. However, maintaining high test coverage proved to be challenging during heavy refactoring of the central implementations and limited the effectiveness of the suite. This is discussed further in the Implementation and Evaluation chapters.

Generative AI tooling was used in two clearly delineated capacities. Firstly, it was used to generate the unit tests from the existing implementations, creating realistic and edge-case test values for the simulation and kinematic components. Secondly, it was used to produce

boilerplate code for the output and visualisation modules with the Matplotlib library — for example axis ticks, coloured legends and subgraph layouts. All simulation, MDP, and kinematic code was written independently. No AI-generated code was included in the main algorithmic implementations, but separate chat windows were used throughout development to aid with documentation searching and reinforcing theoretical knowledge.

Chapter 3

Implementation

3.1 Repository Overview

Path	Description
`src/main.py`	Entry point for individual simulation runs.
`src/tune.py`	Entry point for parameter-tuning simulation runs.
`src/validate.py`	Entry point for behavioural and kinematic validation simulation runs.
`src/simulation/simulation.py`	Simulation kinematics and lifecycle manager.
`src/simulation/mdp.py`	MDP framework, `FastMDP` implementation and reward functions.
`src/outputs/base.py`	Output manager and base output type component.
`src/outputs/*.py`	Output type implementations (e.g. plot, video).
`src/display.py`	Terminal dashboard set-up and lifecycle.
`src/visualisation.py`	`pyvista` interactive 3D visualisation.
`results/`	Directory for any produced outputs.
`src/assets/`	Public-use models for use in 3D visualisation.
`src/configs/`	Configuration values for project components.
`tests/`	Unit tests for project components.

Table 3.1: Repository structure

The simulation and machine learning functionality were coded from scratch without reliance on pre-existing code or external libraries. A selection of features from the Python standard library and mainstream libraries were used:

- `rich` for clear and colourful terminal logs and interfaces.
- `pytest` for unit test management.
- `numpy` for efficient N-dimensional numerical computation.
- `matplotlib` and `asciichartpy` for standard data plotting functionality.
- `pyvistaqt` and `pyqt5` for the interactive 3D visualisation.
- `numba` for JIT compilation of computationally-heavy Python code to machine code at runtime (described further in Extensions).

In all cases, the relevant library provided generic functionality that assisted with performance or visualisation, and additional code was written to interoperate with the simulation implementation.

Before beginning development, the project scaffolding was set up. A `pytest` configuration was set up with a dedicated test folder importing source components, and a GitHub Actions workflow was configured to run the full test suite automatically on every push to the remote repository. The `argparse` standard library was used in a `main.py` file to provide a base command-line interface for configuring and launching simulation runs. The project supported configuration of many key values. Unless otherwise stated, all examples and evaluation in this dissertation use the base configuration included in Appendix C.

Figure 3.1 gives an architectural overview of the final system, explored in detail throughout this chapter.

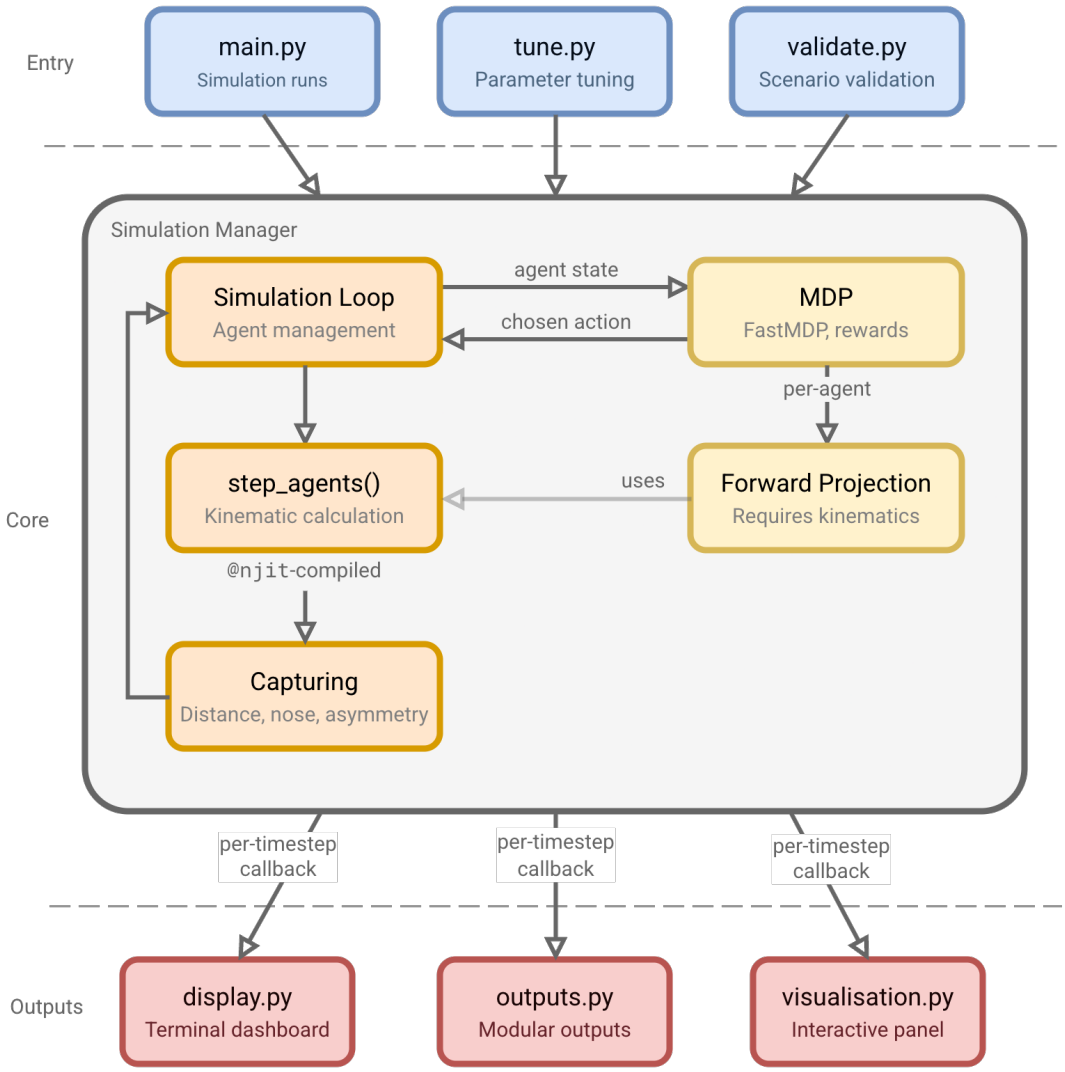


Figure 3.1: System architecture

3.2 Kinematic Model

The kinematic model was developed in incremental steps up to three-dimensionality with pursuit-evasion behaviour expanded at each stage. This allowed for smoother integration of other components (outputs, visualisations), and for the decision-making functionality to be

examined and debugged in much simpler environments before presenting a full solution. The target was a full three-dimensional pseudo-6DoF model as described in the Preparation chapter.

In all cases, time was discretised with a `STEPS_PER_SECOND` configuration value to give a timestep variable $dt = 1/\texttt{STEPS_PER_SECOND}$ for kinematic calculations. Each kinematic value can be updated as $x_{t+1} = x_t + \dot{x} \cdot dt$ through semi-implicit Euler integration for multiple equations, following the discrete-time approach of Bertram et al.[4].

3.2.1 One- and Two-Dimensionality

The aim of one-dimensional kinematics was primarily to validate the simulation timestepping framework and agent system. The space is defined with a right-positive convention and agents are represented with a position and a velocity. The action space is a set of discrete acceleration (m/s^2) values $\{a_1, a_2, \ldots, a_n\}$ with velocity (m/s) updated as $v_{t+1} = v_t + a_{\text{chosen}} \cdot dt$ and position (m) as $p_{t+1} = p_t + v_{t+1} \cdot dt$, all in base SI units. The system interfaced with a naive version of the *FastMDP* algorithm described in section 3.3 and a simple output system, and the resulting trajectories shown in fig. 4.4a confirmed that a decision-making agent approaches its opponent.

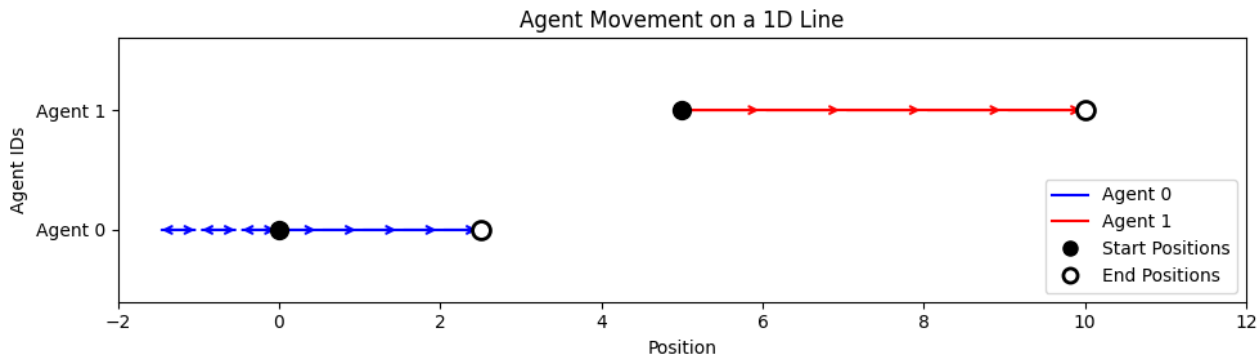


Figure 3.2: 1D Simulation Plot

[REVIEW: do i need to do anymore work on this plot? its just here in its raw state as a development aid, and its effectivness as an visualiastion is considered in Evaluation (spoiler alert: its not great)]

The codebase was initially object-oriented as determined in my design preparation. A `Simulation` singleton maintained a list of `Agent` objects that encapsulated their own kinematic state. This was a natural design given the conceptual clarity of individual agent entities. The decision-making framework also supported this design, as per-agent MDPs could be instantiated from a single class with relevant parameters. A `step()` method could be run each timestep to determine actions, apply them and update kinematic values for each agent.

It became apparent in early development that this design was incompatible with eventual intended optimisations. Individual agent object updates could not take advantage of NumPy's array broadcasting, and Numba's JIT compilation incurred a significant performance penalty when interfacing with arbitrary objects rather than raw procedures.

The aim of two-dimensional kinematics was to more deeply explore the *FastMDP* algorithm and possible reward functions. This would require a base level of optimisation, so I decided to refactor the codebase design to address the above issues. Kinematic calculations were extracted

to a single `step_agents()` method that accepted flat arrays of agent state and applied updates across all N agents simultaneously using NumPy's vectorised operations. This was also a clean interface for later investigating Numba's no-Python JIT compilation mode (described in section 3.5.1).

Two-dimensional kinematics introduced a heading parameter ψ measured in radians, replaced acceleration with thrust value n_x and expanded position and velocity to 2-vectors. The action space became $\{(\dot{\psi}, n_x)\}$, and the numerical integration is shown in listing 1.

```
# input values: heading_rates, thrusts (applied across N agents)
headings = headings + heading_rates * dt

ax = thrusts * np.cos(headings)
ay = thrusts * np.sin(headings)
accelerations = np.stack((ax, ay), axis=-1)

velocities = velocities + accelerations * dt
positions = positions + velocities * dt

return (positions, velocities, headings)
```

Listing 1: 2D kinematic updates via semi-implicit Euler integration using vectorised operations.

The 2D exploration of pursuit-evasion behaviour is described in section 3.3.

[TODO: video artefact of 2d pursuit evasion to include?]

3.2.2 Three-Dimensionality

Three-dimensional kinematics were introduced midway through the project, after a modular output system and the majority of the MDP framework had been implemented, and significantly expanded on the 2D implementation. The agent state grew to include speed V, attack angle α, flight path angle γ, roll angle ϕ, and azimuth angle ψ in addition to position, following the pseudo-6DoF model described in the Preparation chapter. The action space became $\{(n_x, \dot{\alpha}, \dot{\phi})\}$ with thrust n_x, attack angle rate $\dot{\alpha}$ and roll angle rate $\dot{\phi}$ as inputs. The normal force n_f is derived as a speed-dependent G-load.

The equations of motion from Bertram and Wei[4] are implemented directly:

```
max_g_at_speed = np.minimum(MAX_GS, MAX_GS * (velocities / CORNER_VELOCITY) ** 2)
nf = alpha * max_g_at_speed
turn_drag = K_DRAG * (nf ** 2)

speed_rates =
    g *
    (thrusts - turn_drag - np.sin(flight_path_angles))

flight_path_angles_rates =
    (g / speeds) *
    (nf * np.cos(roll_angles) - np.cos(flight_path_angles))

azimuth_angles_rates =
    (g * nf * np.sin(roll_angles)) /
    (speeds * np.cos(flight_path_angles))
```

Listing 2: 3D kinematic rate calculations following the designed pseudo-6DoF model.

Realistic kinematic bounds are enforced through value clipping after integration, with configurable minimums and maximums per-agent to compare parameters.

Parameter	**Range**	**Units**
Speed V	$[34.3, 103.0]$	m/s (0.1–0.3 Mach)
Azimuth rate $\dot{\psi}$	$[-1.3, 1.3]$	rad/s
Attack angle α	$[0.09, 0.52]$	rad (5°–30°)
Flight path angle γ	$(-\pi/2, \pi/2)$	rad
Hard deck	$z > 0$	m
Arena	$10{,}000 \times 10{,}000 \times 10{,}000$	m

Table 3.2: Base kinematic bounds and arena configuration.

The speed and angular state are resolved to a velocity vector for the position update:

```
vx = speeds * np.cos(flight_path_angles) * np.cos(azimuth_angles)
vy = speeds * np.cos(flight_path_angles) * np.sin(azimuth_angles)
vz = speeds * np.sin(flight_path_angles)
velocities = np.stack((vx, vy, vz), axis=-1)
positions = positions + velocities * dt
```

Listing 3: Scalar speed and angles resolved to a 3D velocity vector for position update.

The `step_agents()` function is fully compatible with the `@njit` decorator for Numba's no-Python mode.

The progression from 1D to 3D affected my software methodology throughout implementation. In some cases, heavy refactoring of the codebase design (such as the move away from OOP)

clashed with the agile approach - instead of iterating on previous work and integrating features, structural core changes required additional design work and re-interfacing with existing components. I was still able to adapt to the changing requirements and the incremental progressions had clear functional goals to work towards. Refactoring also impacted the test suite. Tests that had been written against earlier interfaces were invalidated and coverage reduced. The CI pipeline identified these issues and maintaining the tests was not an issue, but this was a limitation of the testing approach that I could evaluate later.

[REVIEW: three dimensional output? video? plot? visualisation screenshot? should that wait until the reward function/decision behaviour is described?]

3.2.3 Capturing

The capturing mechanic tied into the kinematic system. A simulation ends after a maximum number of timesteps, or when all but one agent has been captured. The capturing condition laid out during preparation involved being within a configurable distance of a control point 3 seconds behind an enemy, and aligned towards them within some limit. I found this did not result in accurate-looking capture during examination of the visualisation, so redesigned the criteria during development.

The following must hold simultaneously true for N consecutive timesteps for a capture to occur:

1. **Distance check**: the Euclidean distance between agents falls within a capture radius r.
2. **Nose check**: the pursuer's nose direction is within 50° of the evader.
3. **Asymmetry check**: the pursuer has a meaningfully better alignment than the evader, $a_{\text{pursuer}} > a_{\text{evader}} + 0.2$.

These conditions were designed from scratch and are an original contribution to the problem space investigated in the project. Fragments of implementation can be seen in listing 4. These conditions also guided the eventual design of the reward functions in section 3.3.3.

```
r_pe = evader_position - pursuer_position
d = np.linalg.norm(r_pe)
r_pe_hat, r_ep_hat = r_pe / d, -r_pe / d

pursuer_alignment = np.dot(pursuer_nose, r_pe_hat)
evader_alignment = np.dot(evader_nose, r_ep_hat)

distance_check = d < SimulationConfig.CAPTURE_RADIUS
nose_check = pursuer_alignment >= np.cos(np.deg2rad(50))
asymmetry_check = pursuer_alignment > evader_alignment + 0.2
```

Listing 4: Capture condition values and comparisons

With three-dimensional kinematics and capturing implemented, the core simulation loop was finalised (listing 5). The simulation steps all agents forward by determining their next action and applying kinematic updates. Captures are calculated and applied, out-of-module callbacks are triggered, and ending conditions are checked.

```
while self.simulation.timestep < SimulationConfig.MAX_TIMESTEPS:
    self.simulation.step()
    captures = self.simulation.determine_captures()

    for captured_id, capturer_id in captures:
        all_captures.append(
            (self.simulation.timestep, captured_id, capturer_id)
        )

    for callback in callbacks:
        callback(self.simulation)

    if np.sum(self.simulation.active) <= 1:
        break
```

Listing 5: Simulation loop with capture-breaking logic and per-timestep callbacks

3.3 Decision Making

The decision-making framework is clearly separate from the simulation component, and implements the `FastMDP` algorithm with novel reward functions refined through a parameter tuning process.

3.3.1 `FastMDP`

The full `FastMDP` algorithm was initially implemented alongside two-dimensional kinematics to examine behaviour in a simpler environment. This component remained object-oriented - an `MDP` class is constructed per-agent per-timestep exposing the following methods:

- `find_action` to run the reward-action maximising loop.
- `calculate_rewards` to combine positive and negative rewards into a single surface.
- Two reward functions to compute the positive and negative value functions.
- `hard_deck_penalty` to add a very high negative penalty separate to the parameterised negative rewards when an agent approaches $z = 0$.

At each timestep, before any agent's MDP is solved, all agents are forward projected using zero actions to produce a central world state. Each agent then uses this world state as reward sources - the positions and velocities other agents will reach if they continue on their current trajectory — rather than their current positions. This provides a more accurate reward signal.

Listing 6 shows the implementation of the core decision-making loop. In practice, `current_state` represents the full set of kinematic arrays that are passed individually into the MDP and forward projection process for Numba compatibility, but is simplified here for clarity.

```python
def find_action(self, current_state) -> Action:
    from simulation import forward_project

    num_actions = self.actions.shape[0]
    action_rewards = np.zeros(num_actions)

    # Choose each action
    for i in range(num_actions):
        action = actions[i]

        # Project self agent forward with that action
        projected_positions, projected_velocities =
            forward_project(FORWARD_PROJECTION_STEPS, current_state, *action)

        # Calculate reward from resulting values
        action_rewards[i] = self.calculate_reward(
            projected_positions,
            projected_velocities
        )

    # Choose best action
    best_action = actions[np.argmax(action_rewards)]
    return best_action
```

Listing 6: `FastMDP` action selection via forward projection and reward evaluation.

Listing 7 demonstrates how forward projection was straightforward to implement using the standalone `step_agents` function from the simulation module. The Numba-provided `prange` function allowed `forward_project` to be JIT compiled in the same manner.

```python
@njit
def forward_project(steps, current_state, thrust, azimuth_angle_rate, roll_angle_rate):
    for _ in prange(steps):
        current_state = step_agents(current_state, thrust, attack_angle_rate, roll_angle
    return current_state
```

Listing 7: Forward projection applies an action to a specific agent for a fixed number of steps.

In two-dimensional kinematics, the action space was a finite set of heading turn rates and acceleration values. The reward functions were only position-based and placed positive rewards at enemy positions to drive pursuit.

When I expanded the simulation to 3D kinematics the state and inputs became more complex. The action space constructed as the Cartesian product of configurable ranges of the three kinematic inputs described in the Preparation chapter: thrust, attack angle rate and roll angle rate. These are scaled by ratio values to support heterogeneous agent configurations:

```
self.actions = np.array(
    [
        [
            thrust * thrust_ratio,
            attack_angle * attack_angle_ratio,
            roll_angle * roll_angle_ratio,
        ]
        for thrust in MDPConfig.ACTION_THRUSTS
        for attack_angle in MDPConfig.ACTION_ATTACK_ANGLE_RATES
        for roll_angle in MDPConfig.ACTION_ROLL_ANGLE_RATES
    ]
)
```

Listing 8: Action space generation from configurable ratios.

3.3.2 Parameter Analysis

To inform the design of the final reward functions, a One-at-a-Time sensitivity analysis was performed on the system. A tuning process evaluated the effect of increasing kinematic bound ratios between agents in a 1v1 arena. Starting from an equal base configuration, the script generated linear spaces for each parameter and ran a full simulation until capture or max timestep.

The results of these simulations were visualised in two ways. Figure 3.3 shows a grid of timesteps to capture against parameter value for each investigated parameter individually. Figure 3.4 maps all parameter ranges onto a normalised scale and plots the percentage change in timesteps to capture from the baseline. This allowed a direct comparison of gradients and helped evaluate the dominant parameters.

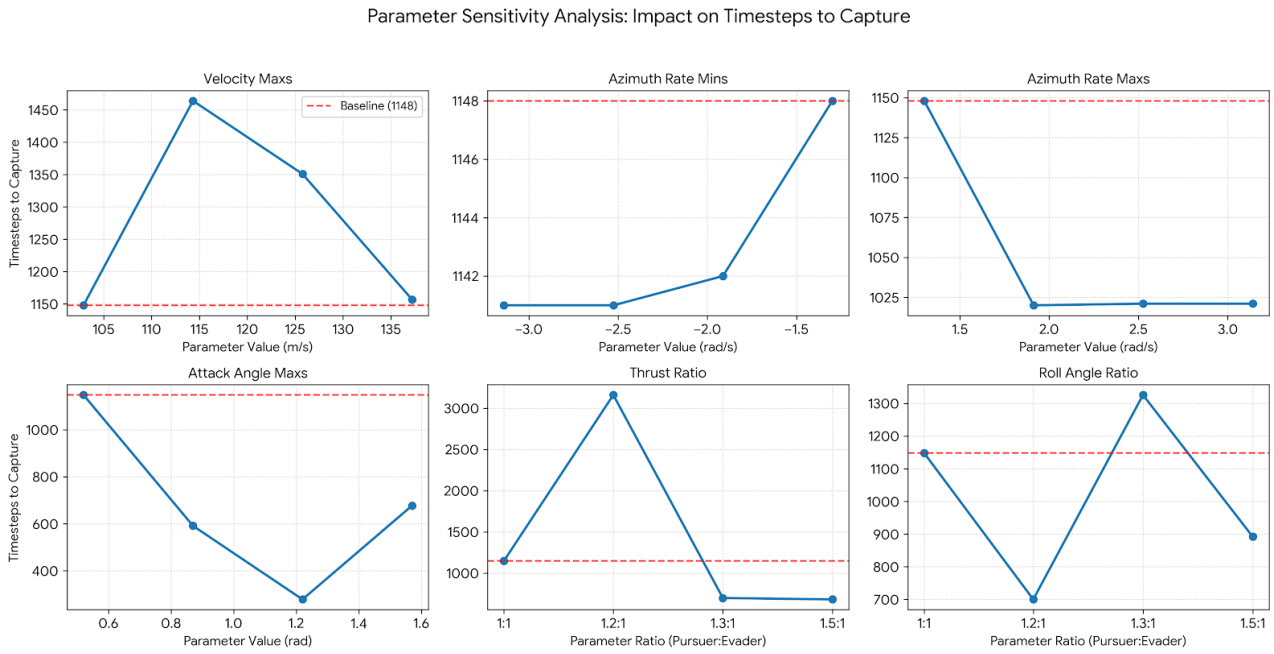


Figure 3.3: One-at-a-Time sensitivity analysis: timesteps to capture against parameter value.

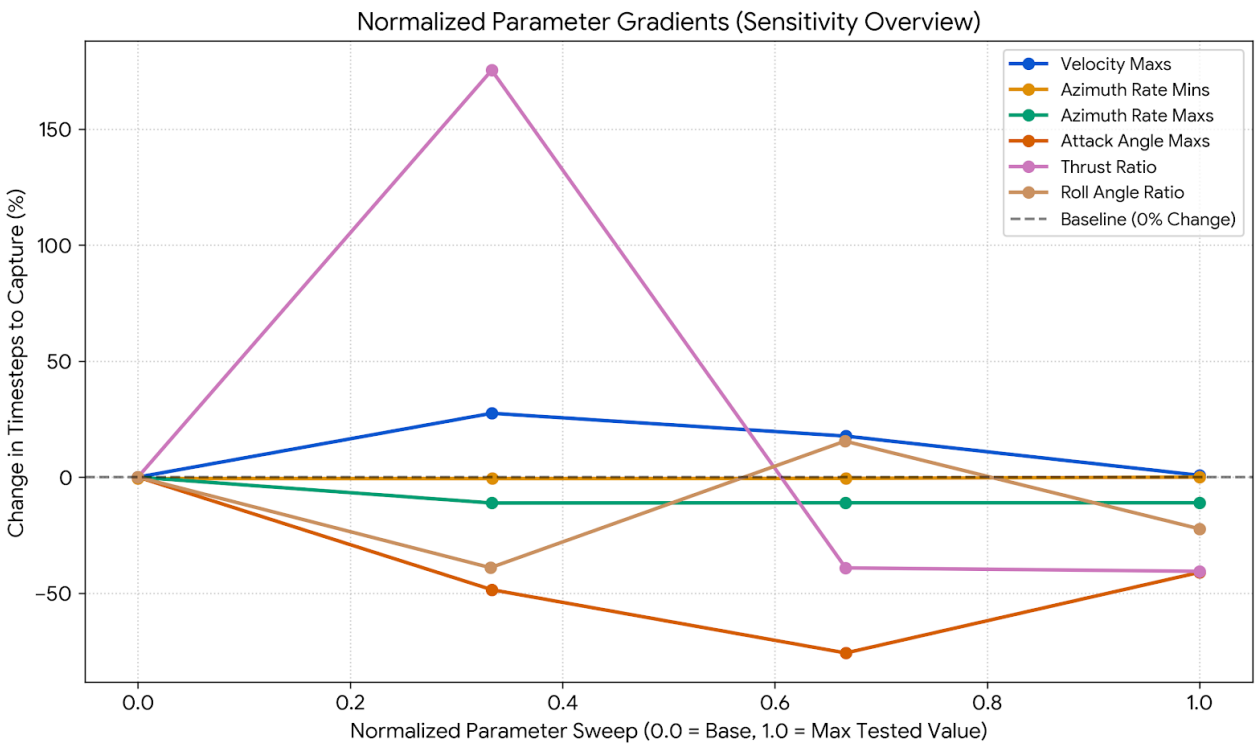


Figure 3.4: Normalised gradients showing percentage change in timesteps to capture from baseline for each parameter investigated.

Attack angle maximum was the dominant parameter, reducing timesteps to capture by approximately 75% at its optimal value — a larger effect than any other parameter. Azimuth rate maximum showed a consistent improvement, confirming that turning capability is a meaningful pursuit advantage. Thrust ratio and velocity maximum showed non-monotonic behaviour, initially increasing timesteps to capture before improving beyond the baseline, which suggested an optimal operating range. Azimuth rate minimum and roll angle ratio had comparatively small effects overall.

These results further motivated the design of alignment-based reward instead of positioning-based ones, as the angular maximum parameters had the greatest effect on pursuit efficiency.

3.3.3 Reward Functions

After investigating the effects of kinematic parameter and bound ratios, reward functions were designed that considered alignment properties when creating the value function surface for the MDP.

```python
def positive_maximum(
    self_position: Vector,
    self_velocity: Vector,
    other_positions: Vectors,
    other_velocities: Vectors,
) -> float:
    r = other_positions - self_position
    d = np.linalg.norm(r, axis=1)
    r_hat = r / d[:, np.newaxis]

    # Self pointing toward enemy
    self_v_hat = self_velocity / np.linalg.norm(self_velocity)
    pursuit_alignment = np.dot(r_hat, self_v_hat)

    # We are in enemy's rear hemisphere
    enemy_v_hat = (
        other_velocities / np.linalg.norm(other_velocities, axis=1)[:, np.newaxis]
    )
    r_from_enemy = -r_hat  # vector from enemy to self
    aspect_alignment = -np.sum(r_from_enemy * enemy_v_hat, axis=1)

    score = (pursuit_alignment + aspect_alignment) / d

    return np.max(score)
```

Listing 9: Positive reward function.

Angular limits had a large effect on pursuit efficiency, so the positive reward function in listing 9 evaluates relative geometry between the agent and opponents. The first term, `pursuit_alignment`, measures alignment between the agent's velocity and the line-of-sight to each opponent to encourage motion towards the target. The second term, `aspect_alignment`, measures the inverse alignment from the opponent and favours being in their rear hemisphere to avoid capture. These terms are summed and inversely scaled by distance to result in the maximal positive reward for close, well-aligned configurations.

```python
def negative_maximum(
    self_position: np.ndarray,
    other_positions: np.ndarray,
    other_velocities: np.ndarray,
) -> float:
    r = other_positions - self_position
    d = np.linalg.norm(r, axis=1)
    r_hat = r / d[:, np.newaxis]

    enemy_v_hat = (
        other_velocities / np.linalg.norm(other_velocities, axis=1)[:, np.newaxis]
    )

    # Calculate alignment
    alignment = np.sum(-r_hat * enemy_v_hat, axis=1)

    # Only penalise if the enemy alignment > 0 (pointing towards self)
    threat_score = np.maximum(0, alignment) / d

    return np.max(threat_score)
```

Listing 10: Negative reward function.

The negative reward function in listing 10 measures the threat from each opponent by considering their orientation towards the agent. This value is also scaled by distance, and clamped to 0 for opponents not actively threatening the agent. The two rewards provide pursuit behaviour through velocity alignment towards a target, whilst simultaneously evading being in a target's line-of-sight. A very large (1×10^9) negative reward is separately applied for any agents below the hard deck of $z = 0$.

```python
best_positive_reward = positive_maximum(*self_state, *others_state)
best_negative_reward = negative_maximum(*self_state, *others_state)
total_reward = best_positive_reward - best_negative_reward
total_reward -= self.hard_deck_penalty(self_position)

return total_reward
```

Listing 11: Calculate final reward.

[REVIEW: is the above listing necessary? could just descirbe in prose how they are summed and hard deck penalty applied]

[REVIEW: possibly place the 3D visualisation video here?]

3.4 Output System

Being able to visualise simulation runs in various ways was a key requirement during development. Qualitative analysis of flight paths, trajectories and agent interactions helped debug

kinematic issues and shape the reward functions.

3.4.1 Plots and Videos

There was a requirement to fully represent the results of a simulation run to inform development and evaluation. Static plots of agent trajectories provided a limited overview, but video outputs allowed them to be observed over time. A modular output manager was implemented that registered a callback with the simulation manager to save agent state each timestep. An abstract `Output` class could be implemented to access this data and save it into various formats.

The `PlotOutput` and `VideoOutput` concrete classes were written to produce simulation visualisations. They worked well, especially for two-dimensional kinematics.

```
for agent_idx, agent_path in enumerate(self.output_manager.agent_paths):
    # Extract positions and stack into (N,3) array
    positions = np.stack([pos for pos, _, _, _, _ in agent_path])

    if len(positions) > 1:
        # Plot the path
        self.ax.plot(
            *positions.T,
            label=f"Agent {agent_idx}",
            linewidth=1.5,
            linestyle=linestyles[agent_idx % len(linestyles)],
        )
```

Listing 12: Plot agent paths on a `matplotlib` 3D graph, discarding unused kinematic data.

[REVIEW: add video artefact? point again to one that was used earlier?]

There were two key limitations to this approach. First, outputs were only produced after a simulation run completed, so there was no way to inspect agent behaviour mid-run. Second, while one- or two-dimensional trajectories could be represented clearly, three-dimensional environments were limited by fixed perspective or camera paths.

3.4.2 Terminal Dashboard

A terminal dashboard was created after implementing the core simulation framework and two-dimensional kinematics to address these issues. It was created with the `rich` library and provided a real-time representation of simulation state. There were two core panels - a display of key values from the simulation at each timestep, and a top-down orthographic projection of agent positions. Additionally there were graphs of CPU usage and memory usage to monitor system metrics throughout the simulation, but these were not fully accurate and were eventually removed in favour of separate performance trackers.

The dashboard (a version of which with the system metrics can be seen in fig. 3.5) was designed to aid rapid iteration and development. Staying in the development terminal allowed me to

run a simulation and immediately sanity check values - e.g initial agent positions and headings or parameter ratios, and its clear layout aided with obvious interpretation. It was primarily a development tool but a good time investment and remained in use until the end of the project.

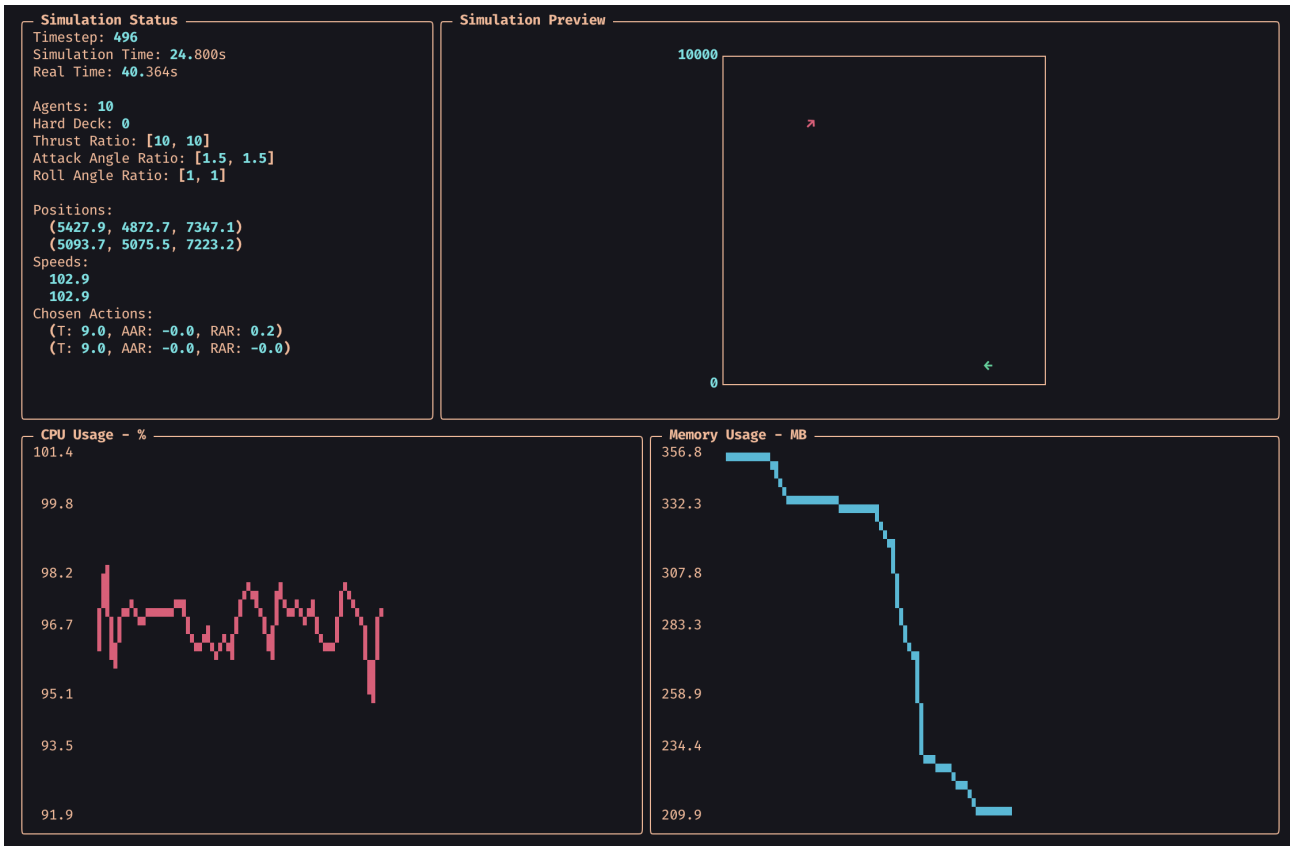


Figure 3.5: Terminal Dashboard

3.4.3 Interactive Visualisation

The final milestone in the output system was a 3D visualisation of the current simulation run implemented with the `pyvista` library, chosen for its interactivity support. It exists inside a `Qt` GUI window which allows for interactive panning, zooming and rotating throughout the simulation run. The application window runs alongside the simulation on a separate thread, registering a callback to update visualised agents (rendered with public-use aircraft models) at each timestep. At close camera ranges, additional velocity direction indicators and speed labels appear by each agent, providing more granular state detail on demand as seen in fig. 3.6. Users can freely rotate their perspective to examine the simulation and agent behaviour.

`pyvista` provided the base platform and interaction functionality, but interoperating code had to be written to add actor meshes to the scene, trigger position updates and manage the rendering loop. A key step involved transforming kinematic angles from the simulation space to the world space of the visualisation, as seen in listing 13.

```python
def get_pyvista_orientation(
    attack_angle: Scalar,
    flight_path_angle: Scalar,
    azimuth_angle: Scalar,
    roll_angle: Scalar,
) -> tuple[float, float, float]:
    pitch = flight_path_angle + attack_angle
    pitch_deg = np.rad2deg(pitch)
    yaw_deg = np.rad2deg(azimuth_angle)
    roll_deg = np.rad2deg(roll_angle)

    return (roll_deg, -pitch_deg, yaw_deg)
```

Listing 13: Convert simulation angles to `pyvista` visualisation space.

This was the final visual output of the project and information design principles were applied to ensure a high-quality visualisation, examined further in the Evaluation chapter.

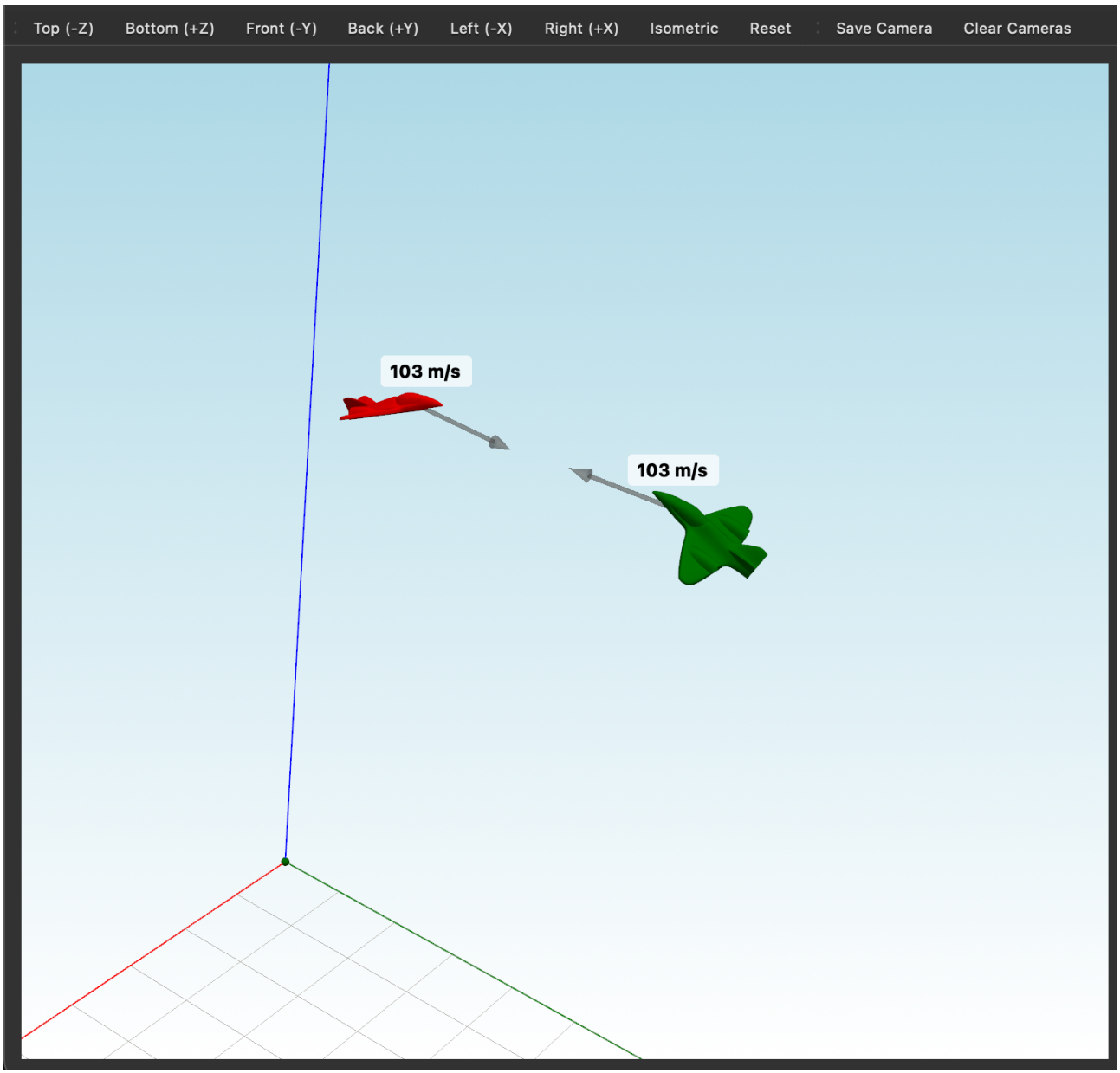


Figure 3.6: 3D Simulation Visualisation (with additional velocity details)

3.5 Extensions

3.5.1 Numba Compilation

Numba's JIT compilation was investigated as a performance optimisation for the kinematic calculations in section 3.2.2. This compilation happens on the first call to the function, and subsequent calls need not access the Python interpreter at all, often resulting in significant performance gains. Numba is specifically designed for numerical computation and integrates with NumPy arrays well, making it well suited for the vectorised kinematics of this project. It provides two modes: `@jit` object mode, which attempts to compile Python code but can fall back to slow interpreted object access if necessary, and `@njit` no-Python mode, which forces full compilation to machine code and raises an error for unsupported constructs.

No-Python mode was my target as it provides the greatest performance gains, but it imposes constraints on what code can be compiled. Python objects, class instances and much of the standard library are not supported - only NumPy arrays and a subset of NumPy operations are allowed. This motivated design shifts away from OOP as previously discussed, but also demonstrated a trade-off between performance and maintainability. In the original `step()` function scalar-array polymorphism was supported, and it could accept either a single agent's state or an array of N agent states. This was exploited in the MDP's forward projection, where only the ownship's state needed to be projected forwards. With an `@njit` decorator this functionality was sacrificed, and the MDP was forced to construct or slice single-element arrays for each scalar as demonstrated in listing 14.

```
# Kinematic calculation agent for arrays of N agents
# Used in simulation to step all agents forwards at once
@njit
def step_agents(positions, speeds, ...):
    # equations of motions...

# MDP forward projection
# This fails in @njit - scalar doesn't broadcast.
step_agents(positions[i], speeds[i], ...)

# Fix - explicit single-element slice maintains dimensionality
step_agents(positions[i:i+1], speeds[i:i+1], ...)
```

Listing 14: Scalar-array polymorphism trade-off for `@njit` broadcasting consistency.

The performance impacts of Numba integration are investigated in the Evaluation chapter.

3.5.2 Multi-Agent Runs

3.5.3 Alternative Behaviours

[REVIEW: mark scheme says contribution to the field with genuine impact. should I state that stuff that in this chapter? i originally had some stuff about it in the conclusion.]

Chapter 4

Evaluation

I revisited the success criteria outlined in the Introduction chapter to inform evaluation. I undertook quantitative measurement and qualitative analysis of various components of the system to guide critical interpretations of success. The evaluation includes simple cases to demonstrate base functionality alongside more ambitious and edge-case configurations to expose limitations.

1. **Behavioural validity**. The implemented FastMDP algorithm produces valid pursuit-evasion behaviour, defined according to a series of per-timestep constraints.
2. **Kinematic validity**. Agent kinematics remain within physically realistic bounds throughout all simulation runs, including speed, turn rate and altitude constraints.
3. **Computational performance**. Agent actions are computed within a timeframe suitable for real-time simulation with appropriate optimisations investigated and evaluated.
4. **Extension validity**. Multi-agent teams and alternative problem spaces produce valid and consistent behaviour under the same framework.

A showcase of the final simulation is presented below. The scenario is a mid-complexity 1v1 encounter to demonstrate airflight simulation and interacting pursuit-evasion behaviour between agents.

[TODO: ADD VIDEO]

4.1 Functional Evaluation

Throughout implementation, I had been iteratively adjusting the FastMDP algorithm and reward functions to drive desirable behaviour. As described in the Implementation chapter this process was driven partly by qualitative analysis of the flight paths and partly by sensitivity analysis of the kinematic parameters. After completing the project core, I wanted to more strictly examine the pursuit and evasion behaviour of the agents to inform the success of the project.

I designed a set of 1v1 validation scenarios, each defined by a specific combination of initial values and agent parameter ratios, ranging in complexity. Each scenario was run for up to 1000 timesteps or until a capture, and at each timestep the agent values were recorded and checked against predefined validation criteria. A scenario is considered valid if all tracked values satisfy these criteria throughout the run. The exact values of the scenarios and their validation results can be found in Appendix C.

4.1.1 Validity Definitions

I defined two types of validity and the criteria for them between timesteps.

Kinematic Validity

I established a set of three kinematic parameters and their realistic ranges.

- Speed $s \in [34.3, 103.0]$ m/s (0.1 - 0.3 Mach)
- Azimuth rate $\dot{\psi} \in [-1.3, 1.3]$ rad/s
- Altitude $z > 0$ m

A scenario is considered kinematically valid if these hold for all agents at all timesteps.

Behavioural Validity

Let $d(t) = ||\mathbf{p}_1(t) - \mathbf{p}_2(t)||$ be the Euclidean distance between a pair of agents with position functions $\mathbf{p}_1$ and $\mathbf{p}_2$.
Further let $d_0 = d(0)$ the initial separation and $d_{\min} = \min_{t \in [0,1000]} d(t)$ the minimum separation of the run.

A scenario is considered behaviourally valid if:

1. $\exists\, t.d(t) < d(t-1)$ — i.e. $d(t)$ does not monotonically increase throughout the run.
2. $d_{\min} < d_0$ OR capture occurs during the run.

Together these ensure that the reward functions produce meaningful interaction — a non-increasing $d(t)$ at some point implies that at least one agent is closing on the other's position, and $d_{\min} < d_0$ confirms this results in net closure over the course of the run.

If all agents are kinematically valid, and all pairs of agents are behaviourally valid, then the scenario is valid.

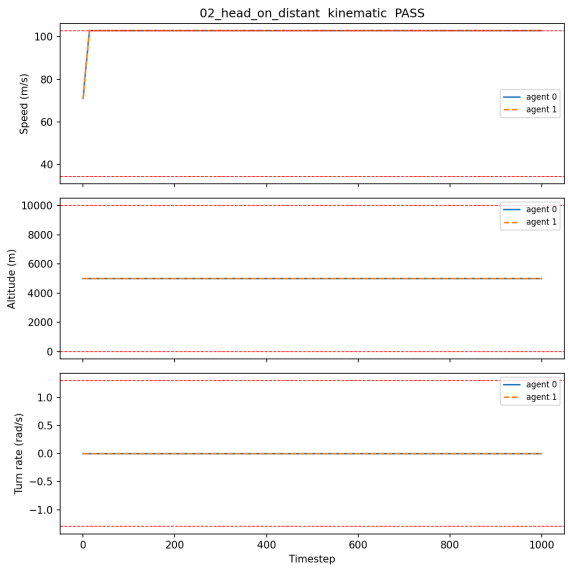
4.1.2 Observations

Each scenario was run and the kinematic and behavioural validity criteria evaluated at each timestep. Timestep-series plots were produced to demonstrate key values and properties for each validity.

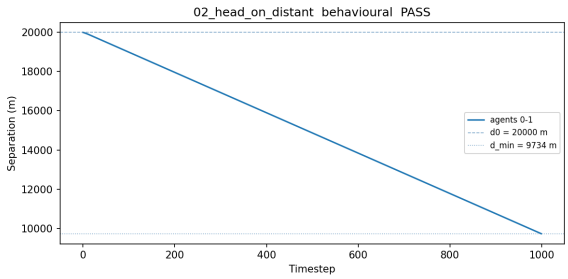
The symmetric scenarios (1, 2, 3, 5) produce the same kinematic plots for both agents, which is expected — identical initial values and MDP parameters should produce identical behaviour, and any asymmetry would indicate an unintended bias.

Scenario 1 is the simplest case — the agents are initialised head-on at large separation, and we see them thrust up to maximum speed in fig. 4.1a and their distance decrease monotonically in fig. 4.1b. This scenario primarily validates kinematic correctness rather than complex decision-making as straight-line flight could also be achieved in the absence of any actions.

A more interesting example is scenario 6, designed to have the most asymmetric initial values. The scenario ends with a capture at timestep 712, expected when one agent begins in a more favourable position. Quantitatively we can see that the kinematic and distance values meet the

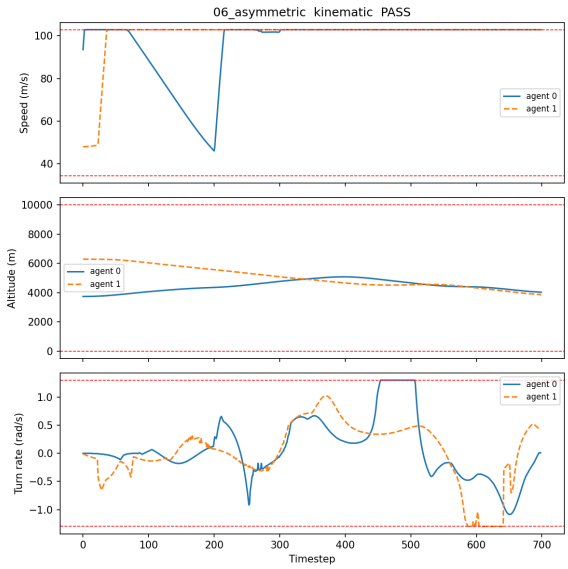


(a) Kinematic Values

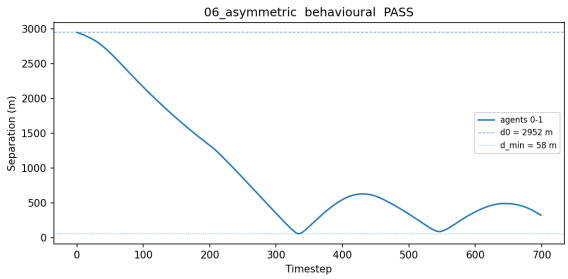


(b) Behavioural Values

Figure 4.1: Scenario 1 (facing head-on, equal speed, large separation).



(a) Kinematic Values



(b) Behavioural Values

Figure 4.2: Scenario 6 (facing ∼53° off-axis, 14:26 speed ratio, lateral and altitude separation).

criteria for validity even with complex manoeuvres and arbitrary initial conditions. Qualitative examination indicates periods of overcorrection, shown in the altitude crossovers from timestep ∼330 onwards in fig. 4.2a and the separation bounces in fig. 4.2b.

This overcorrecting fits with two interacting components of the system. The positive and negative rewards are based on alignment and scale inversely with distance, so they interact to encourage closing-in encircling where agents approach each other without offering a capture. At close ranges, agents can exploit this by changing their own velocity alignment to reposition their opponent in their front or rear hemisphere, which is an unexpected limitation in the reward function structure that leads to the bouncing-away effect. Additionally, the short forward projection horizon means the MDP cannot anticipate the full trajectory, so agents thrust up to full speed to approach faster but cannot correct in time before overshooting.

In scenario 3, agents are initialised facing directly away from each other. Rather than immediately maximising their turn rates towards each other, they use a combination of lift and more gradual turns. This is consistent with the reward function structure — agents initialised facing away are already in each other's rear hemispheres which gives higher reward, so the MDP favours slower reorientation whilst closing distance.

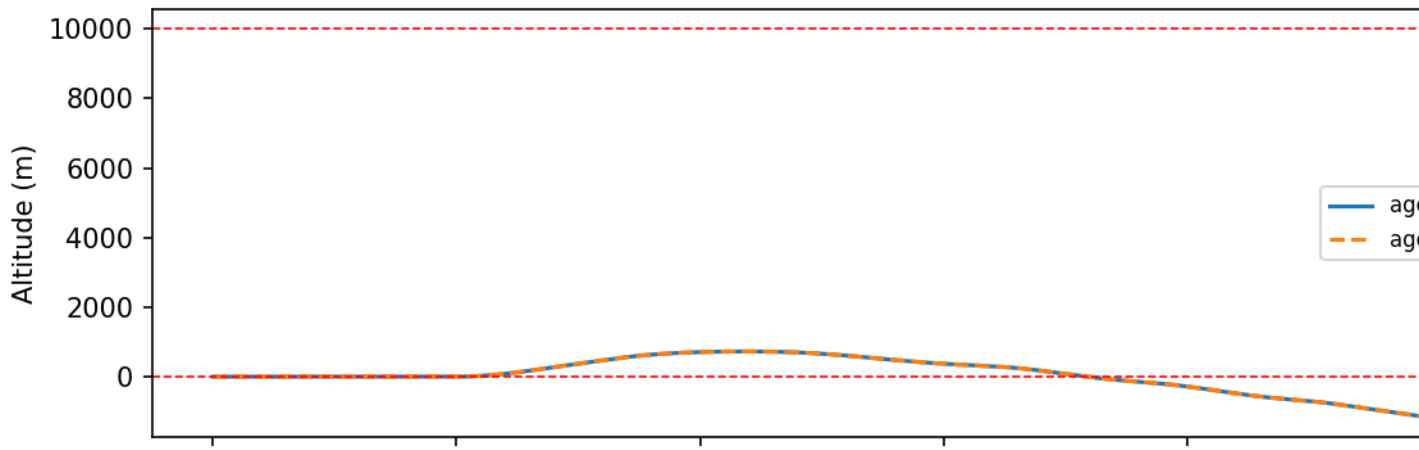


Figure 4.3: Scenario 7 (facing head-on, equal speed, medium separation, low altitude).

Scenario 7 failed kinematic validity — the agents dipped below the hard deck of $z = 0$m at timestep ∼700. This is a reward function failure, as the hard-coded hard-deck penalty did not prevent the altitude drop. It is likely not an issue of weight as the penalty (1×10^9) is greater than the standard rewards.

4.1.3 Results Summary

All scenarios passed the behavioural validity criteria, and all but one passed kinematic validity (with scenario 7 failing the hard-deck constraint as described in section 4.1.2). The implemented reward functions and parameter tuning drive acceptable pursuit-evasion behaviour, but closer examination of key parameters (speed, turn rates, separation) revealed limitations with the positive and negative rewards.

Since the reward functions are mostly alignment-based, turning artefacts are observed especially at close ranges. The positive reward

$$R^+ = \frac{\hat{v}_{self} \cdot \hat{r} + \hat{r} \cdot \hat{v}_{enemy}}{d} \tag{4.1}$$

is maximised when the agent points towards the opponent and the opponent points away. The negative reward

$$R^- = \max\left(0, \frac{(-\hat{r}_{pe}) \cdot \hat{v}_{enemy}}{d}\right) \tag{4.2}$$

penalises configurations where an opponent is actively oriented towards the agent. They both scale with $1/d$, so at close ranges small changes in velocity direction produce large changes

in reward, incentivising sharp movements rather than smooth adjustments. Agents can also increase their positive rewards by adjusting their own alignments relative to their opponent, producing the bouncing behaviour observed in scenarios 3 and 6.

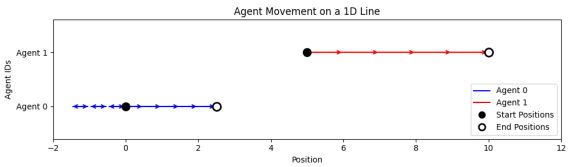
The hard-deck penalty did not prevent altitude violations, possibly due to the forward projection horizon not considering states near $z = 0$ and not encountering the penalty during planning.

4.2 Engineering Performance

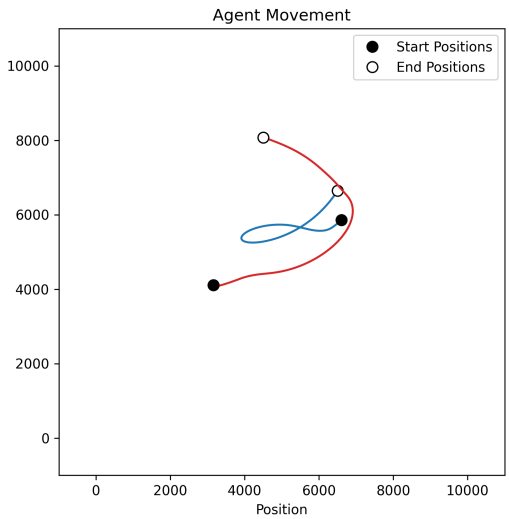
Talk about Numba prange and parallelTrue.

4.3 Visualisation Evaluation

The output and visualisation systems are evaluated with reference to Tufte's ideas across *The Visual Display of Quantitative Information*[13] and *Envisioning Information*[14]. Throughout the project the aim was to produce results that could inform quantitative and qualitative analysis across implementation and evaluation.



(a) 1D Simulation Plot



(b) 2D Simulation Plot

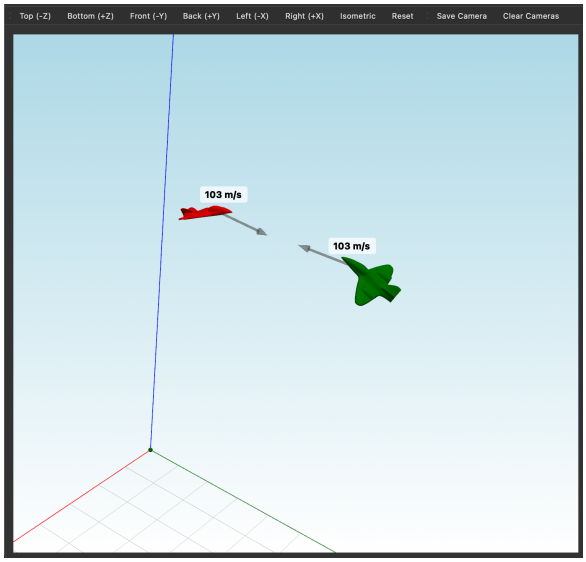
Figure 4.4: Simulation plots for one- and two-dimensional kinematics.

[REVIEW: I need to do a bit more work on these plots possibly? - axis labels, maybe background grid lines, colour legends for 2d... but still evaluate as a failure of principle application]

Static plots were produced for one and two-dimensional kinematics. Whilst suitable, the outputs did not fully apply Tufte's layering and separation principle[13] — in particular the 1D plot direction arrows were not clearly separable from the movement lines (fig. 4.4a). This is a consequence of the plots being quickly-produced development aids rather than full results.

The initial three-dimensional plot (fig. 4.5a) illustrated Tufte's *Escaping Flatland*[14] challenge for representing high-dimension data. Even transforming this into a video format to animate trajectory over time was still limited by a fixed camera perspective that limited spatial understanding **[REVIEW: need a link to a video artefact of this?]**. Instead, I turned to Shneiderman's mantra to implement the interactive visualisation — "overview first, zoom and filter, then details on demand" [15]. The default view provides an *overview* of the full arena, but the user can *zoom* and pan to *filter* to particular agents, and additional velocity *details* appear at close-enough ranges. This addressed the flatland problem by allowing the user to freely reposition their orientation instead of being limited to a single perspective. The velocity details also support Tufte's Micro/Macro readings, where the relative agent positions are primarily readable in the overview perspective and more granular details become available on zoom.

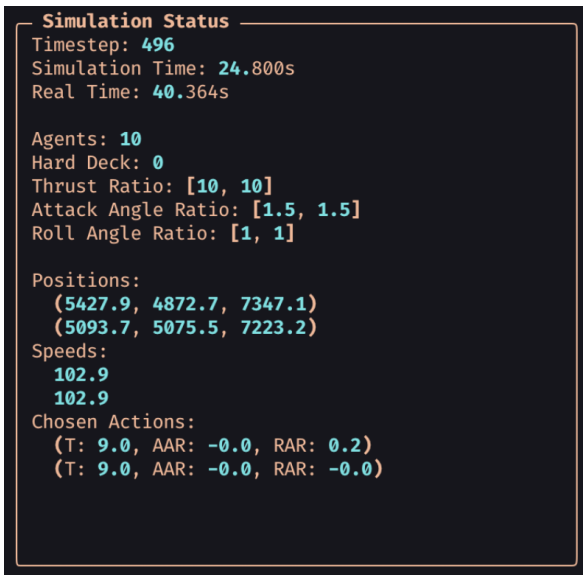
(a) 3D Simulation Plot



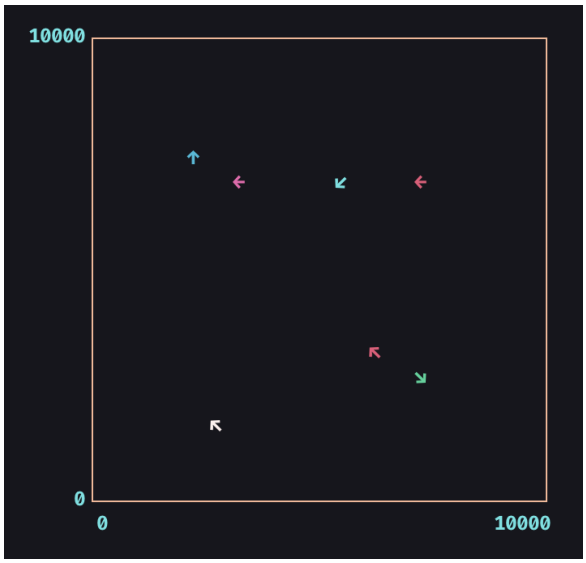
(b) 3D Simulation Visualisation (with additional velocity details)

Figure 4.5: Three-dimensional simulation output: static plot and interactive visualisation.

The terminal dashboard provides real-time data throughout simulation runs. The *Values* panel (fig. 4.6a) shows key simulation state and uses colouring principles to separate labels from values. This was useful for comparing agent values at different points in the simulation. The lightweight *Preview* panel (fig. 4.6b) provides a two-dimensional overview of agent positions and azimuth angle headings. This is a compromise on the flatland problem: key kinematic properties are projected orthographically into two dimensions to trade spatial accuracy with development speed. For example, this preview allowed for immediate verification of correct initial headings for the validation scenarios used in section 4.1 without the computational load of running the interactive visualisation. Each agent's symbol is represented with a different colour, further implementing Tufte's colouring principles to draw clear distinctions between entities.



(a) *Values* Panel



(b) *Preview* Panel

Figure 4.6: Terminal dashboard panels showing real-time simulation state.

4.4 Extension Validity

4.5 Software Methodology Evaluation

4.5.1 Testing Strategy

The GitHub actions CI testing pipeline was robust and ran automatically on every remote commit. Using generative AI allowed for easy iteration of tests to match system refactors, but this process was implementation-led rather than test-driven.

Coverage suffered during periods of heavy refactoring, especially when trying to introduce optimisation libraries including `numpy` and `numba`. These changed the core structure of the code, and invalidated existing tests because they no longer matched interfaces or mocked objects. Further consideration could have been taken when designing module interfaces, or planning the progression from 1D to 3D simulation to ensure that a test suite could evolve alongside the codebase and provide more value.
The typical workflow was to implement a component or refactor an existing module, verify it through manual program runs, and then generate tests to confirm its behaviour. Tests became indicators of previous correctness rather than the basis for refactors and new functionality.

The pipeline did catch actual issues — for example, a failing run on commit `68e85c6` identified that the output manager did not support the newly-integrated pyvista visualisation outputs, but overall a key lesson learnt was to take a more test-driven methodology.

[REVIEW: was planning on making a coverage graph over time but GitHub has deleted my "historical" test report build artifacts :( ]

4.5.2 Agile Effectiveness

Three-week agile sprints guided the development schedule with short reviews of completed functionality written after each sprint. The core benefit was iterative expansion, where it helped me stay on track to implement base functionality whilst reacting to changing requirements in the codebase. An example of this was the `numba` library integration, where I dedicated time to adjust the central simulation framework to procedural methods before reviewing the work and continuing with other features.

Before beginning a sprint I used the previous review to guide functionality for the next one, but was flexible in considering different modules or components that made the most sense to implement next. The reviews show a clear progression of design decisions towards the success criteria of the project.

To further improve the effectiveness of the methodology I could have incorporated a ticketing system to track the implementation progress of more granular features, but as a solo developer with a comprehensive design plan for the codebase I felt confident with the methodology and its flexibility.

The lessons learnt from this project and subsequent evaluation are considered in the Conclusion chapter.

Chapter 5

Conclusions

5.1 Summary of Work

This project investigated, implemented and extended the FastMDP algorithm of Bertram and Wei[4] in a three-dimensional kinematic simulation of fixed-wing aircraft, with a primary focus on pursuit-evasion behaviour. A pseudo-6DoF kinematic model was implemented incrementally from one to three dimensions, with functionality for parameter variation and limit adjustments. New alignment-based reward functions were designed and refined through one-at-a-time sensitivity analysis, and original capturing conditions were contributed to the problem space. The project was evaluated against defined kinematic and behavioural validity criteria, performance requirements and visualisation principles.

An output system of static plots, video outputs, a terminal dashboard and an interactive 3D visualisation was developed throughout to support analysis and evaluation. The project was extended with Numba JIT compilation for performance gains, multi-agent scaling beyond 1v1, and a demonstration that the decision-making framework generalises to alternative problem spaces including **[TODO: what spaces]**

5.2 Lessons Learnt

The project expanded my knowledge across several areas. I learnt foundational MDP theory and approximate solution methods before implementing the FastMDP algorithm. Designing reward functions to drive behaviour developed analytical thinking — examining quantitative and qualitative flight data helped me consider how to encode tactics into mathematical relationships, and using sensitivity analysis gave empirical justifications for the final result.

From a software engineering perspective, the value and limitations of my testing approach became apparent during periods of heavy refactoring. The progression from OOP to vectorised NumPy to Numba-compiled procedures had design issues that were not anticipated during preparation. Tests written against earlier interfaces were repeatedly invalidated, and the key lesson was that test-driven development with stable interfaces would have provided more value than the implementation-led approach taken.

My evaluations helped me consider principles of information and visualisation design. The project provided an environment to explore how abstractly-implemented simulations can be concretely visualised and intuitively explored, especially with interactive elements for end users.

If I started the project again, I would place more emphasis on test-driven development and design module interfaces in-depth before their implementations. I could also consider multicore and GPU parallelism in the design process, as the action-space evaluation loop in FastMDP is highly parallelisable and could benefit from these approaches.

5.3 Contributions to the Field

The project makes several contributions beyond reimplementing existing work. The capturing conditions that combine distance, nose alignment and asymmetry checks are an original contribution not present in Bertram and Wei[4], and provide a meaningful terminating condition for the simulation. The alignment-based reward functions extend position-based approaches by considering velocity geometry, and the one-at-a-time sensitivity analysis compared the relative importance of different kinematic parameters.

The pseudo-6DoF simulation, with vectorised numerical computation and JIT-compilation with Numba, is a reusable component independent of the pursuit-evasion problem space. I demonstrated that the decision-making framework can also be generalised to other behaviours with changes only to the reward function, indicating applicability of the `FastMDP` algorithm outside of the pursuit-evasion space. **[TODO: what spaces]**

There are ethical considerations around the problem space of autonomous air combat. Weapon systems that make engagement decisions without human input raise concerns about accountability and the barriers of entry to combat. The framework developed in the project exists in a simulated environment with no path to actual deployment, but further formation could contribute to research in this area and the implications should be acknowledged. As noted, applications to other spaces **[TODO: have these benefit...]**

5.4 Future Work

Further refining could be performed on the reward functions to address some of the artefacts identified during evaluation. The distance-bouncing could be addressed by incorporating closure-rate terms or even increasing the forward projection horizon. The hard deck penalty did not prevent altitude violations in all cases and a harsher behavioural block may be required.

Parallelising the action-space evaluation loop across CPU cores or GPU threads could present a significant performance improvement. To further improve the native gains from Numba, the core computation components of the project could be rewritten in a lower level language such as Rust and communicate back with the visualisation components through foreign function interfaces.

Bibliography

[1] Isaac E. Weintraub, Meir Pachter, and Eloy Garcia. "An Introduction to Pursuit-Evasion Differential Games". In: *arXiv preprint* (2020). arXiv: 2003.05013.

[2] James S. McGrew et al. "Air-Combat Strategy Using Approximate Dynamic Programming". In: *Journal of Guidance, Control, and Dynamics* 33.5 (2010), pp. 1641–1654.

[3] Josh Bertram et al. "Online Flight Planner with Dynamic Obstacles for Urban Air Mobility". In: *AIAA Aviation 2019 Forum*. 2019. DOI: 10.2514/6.2019-3625.

[4] Jason Bertram and Jianghai Wei. "Scalable Online Planning for Multi-Agent MDPs". In: *arXiv preprint* (2019). arXiv: 1909.04171.

[5] Rufus Isaacs. *Differential Games*. Wiley, 1965.

[6] Dimitri P. Bertsekas and John N. Tsitsiklis. *Neuro-Dynamic Programming*. Athena Scientific, 1996.

[7] Warren B. Powell. *Approximate Dynamic Programming: Solving the Curses of Dimensionality*. Wiley, 2007.

[8] Richard Bellman. *Dynamic Programming*. Princeton University Press, 1957.

[9] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 1998.

[10] Byung-Yoon Lee et al. "One-versus-one air-to-air combat maneuver generation based on differential game". In: *Proceedings of the 30th Congress of the International Council of the Aeronautical Sciences (ICAS)*. 2016.

[11] H. Huynh, Ph. Costes, and C. Aumasson. "Numerical optimization of air combat maneuvers". In: *AIAA Guidance, Navigation and Control Conference*. 1987. DOI: 10.2514/6.1987-2392.

[12] Numba Developers. *Installing Numba*. 2024. URL: https://numba.readthedocs.io/en/stable/user/installing.html (visited on 04/21/2026).

[13] Edward R. Tufte. *The Visual Display of Quantitative Information*. Graphics Press, 1983.

[14] Edward R. Tufte. *Envisioning Information*. Graphics Press, 1990.

[15] Ben Shneiderman. "The Eyes Have It: A Task by Data Type Taxonomy for Information Visualizations". In: *Proceedings of the IEEE Symposium on Visual Languages*. 1996, pp. 336–343.

Appendix A

Sprint Reviews

Sprint Reviews

The following are the raw sprint reviews written throughout the project, kept in their original form as development notes.

Sprint 1 — Oct 3–20 2025

- Set up initial project structure and repository organisation
- Python virtual environments, CLI framework

Sprint 2 — Nov 6 2025

- Implemented simulation agents and an output manager supporting both plot and video output formats
- Added terminal display and genAI basic unit tests for agent and simulation components

Sprint 3 — Dec 22–29 2025

- Investigated numba integration for simulation acceleration
- Added positive/negative reward functions & refactored simulation internals for dimensionality increases

Sprint 4 — Jan 12–22 2026

- Implemented full MDP with reward parameter tuning and simplification passes
- Extended visualisation to 3D - updating simulation & outputs

Sprint 5 — Feb 6–12 2026

- PyVista for interactive simulation rendering, investigated visualisation improvements
- Live visualisation output with velocity-based agent behaviour and a capturing mechanic

Sprint 6 — Mar 31 2026

- Tuning process implemented and carried out, dominant parameters identified
- Miscellaneous improvements across the codebase

Sprint 7 — Apr 10–12 2026

- Set up a validation pipeline and split the codebase into main/tune/validate components
- Updated tests to reflect the new structure

Appendix B

Configuration Values

The following table lists all configuration values used in the base project configuration.

Parameter	Value	Description
Simulation (`configs/simulation.py`)		
`MAX_TIMESTEPS`	4000	Maximum timesteps per simulation run.
`AGENTS`	7	Number of agents in the simulation.
`STEPS_PER_SECOND`	20	Timesteps per simulated second.
`HARD_DECK`	0 m	Minimum altitude.
`WIDTH / HEIGHT / LENGTH`	10,000 m	Arena dimensions.
`PENALTY`	1×10^9	Hard deck reward penalty.
`CAPTURE_RADIUS`	500 m	Distance threshold for capture.
`CAPTURE_POINT_STEPS`	40	Timesteps for capture point history.
`MAX_GS`	9.0	Structural G limit.
`CORNER_VELOCITY`	80.0 m/s	Speed at which maximum G becomes available.
`K_DRAG`	0.05	Induced drag coefficient.
`MACH`	343 m/s	Speed of sound.
MDP (`configs/mdp.py`)		
`FORWARD_PROJECTION_STEPS`	3	Steps used in forward projection.
`ACTION_THRUSTS`	$[0.0 - 0.9] \times 10$	Thrust action values.
`ACTION_ATTACK_ANGLE_RATES`	$[-1.0 - 0.8] \times 10$	Attack angle rate action values.
`ACTION_ROLL_ANGLE_RATES`	$[-1.0 - 0.8] \times 10$	Roll angle rate action values.
Agent Parameters (`configs/parameters.py`)		
`velocity_mins`	0.1 Mach	Minimum airspeed per agent.
`velocity_maxs`	0.3 Mach	Maximum airspeed per agent.
`azimuth_rate_mins`	-1.3 rad/s	Minimum azimuth rate per agent.
`azimuth_rate_maxs`	1.3 rad/s	Maximum azimuth rate per agent.
`attack_angle_mins`	0.09 rad	Minimum attack angle per agent.
`attack_angle_maxs`	0.52 rad	Maximum attack angle per agent.
`thrust_ratio`	10.0	Thrust action scaling per agent.
`attack_angle_ratio`	1.5	Attack angle rate scaling per agent.

Parameter	Value	Description
`roll_angle_ratio`	1.0	Roll angle rate scaling per agent.
Output (`configs/output.py`)		
`DPI`	300	Plot output resolution.
`FIGSIZE`	(6, 6)	Plot figure size in inches.

Table B.1: Base configuration values.

Appendix C

Validation Scenarios

The following scenarios were used for functional evaluation in section 4.1.

ID	Separation	Heading	Velocity Ratio	Altitude
1	Close (∼500m)	Head-on	Equal (0.2 vs 0.2 Mach)	Mid (5000m)
2	Distant (∼20km)	Head-on	Equal (0.2 vs 0.2 Mach)	Mid (5000m)
3	Mid (∼2000m)	Facing away	Equal (0.2 vs 0.2 Mach)	Mid (5000m)
4	Mid (∼2000m)	Perpendicular	Equal (0.2 vs 0.2 Mach)	Mid (5000m)
5	Mid (∼2000m)	Head-on	Asymmetric (0.3 vs 0.15 Mach)	Mid (5000m)
6	Asymmetric	∼53° off-axis	Asymmetric (0.26 vs 0.14 Mach)	Asymmetric (3740m vs 6290m)
7	Mid (∼2000m)	Head-on	Equal (0.2 vs 0.2 Mach)	Low (15m)

Table C.1: Validation scenario configurations.

Appendix D

Proposal

D.1 Introduction

Pursuit-Evasion is a problem domain in which one or more pursuers attempt to capture one or more evaders in some environment. This field has applications and links to game theory, spatial algorithms and air flight kinematics. In this project I will consider a three-dimensional continuous space populated by aerodynamically-accurate entities and investigate how a Markov Decision Process (MDP) can be used to model and solve pursuit-evasion problems in this space.

The core of my project will incorporate work from [1] where I will implement their `FastMDP` algorithm for efficiently solving pursuit-evasion problems using a Markov Decision Process. I will extend this work by considering how I can improve the visualisation of the simulation, consider additional kinematic and environmental constraints and introduce additional agents to the problem.

D.2 Structure of the Project

Project Core

The project core can be broken down into the following components. For each component I describe the work required and my implementation approach.

The project will be implemented in Python 3.13.

Visualisation

Initially the program's output will be a static image showing the paths taken by the pursuers and evaders by plotting their positions at each timestep. This is sufficient to demonstrate the basic functionality of the simulation. I will use the python library `matplotlib` to create these plots.

Modelling Agents Kinematically

Each agent will be modelled using a pseudo-6DoF (degrees of freedom) kinematic model inspired by the formulation in [1]. Each agent includes parameters:

- V: Airspeed in metres per second
- (x, y, z): Current position
- γ: Flight path angle in radians
- ϕ: Roll angle in radians
- ψ: Yaw angle in radians
- α: Angle of attack in radians

The inputs to the model will be the thrust n_x, the rate of change of ϕ, and the rate of change of α. The equations of motions of the agents can be calculated from these parameters and used for forward projection in the MDP.

Markov Decision Process and `FastMDP` Algorithm

Markov Decision Processes (MDPs) are a mathematical framework for decision-making in situations where outcomes are partly random and partly under the control of an agent. In pursuit-evasion, each agent's position and velocity define the state, possible manoeuvres define the actions, and the environment responds probabilistically. The `FastMDP` algorithm described in [**BertramWei2021**] provides an efficient way to solve pursuit-evasion problems using MDPs using forward projection of the simulation and policy evaluation.

Extensions

I will consider several possible extensions to the project core that explore additional constraints or complexities in the problem space. These include:

- Introducing terrain and obstacles into the 3D environment.
- Adding additional agents and investigating collaborative strategies among them.
- Implementing advanced aerodynamic limitations, such as stalling and maximum turn rates.
- Creating a dynamic visualisation, outputting a video or interactive 3D model of the simulation at each timestep and representing the agents visually with 3D models.

D.3 Starting Point

I have some experience with using and building Markov models in Python (in particular Markov Chains and Hidden Markov Models) from my Part IA Machine Learning and Real-World Data course, but I have not specifically implemented a Markov Decision Process or the `FastMDP` algorithm before.

I do not have experience with pursuit-evasion problems or explicit kinematic models, but I have written Python code to simulate physical three-dimensional systems before with a focus on linear algebra and vector mathematics rather than aerodynamics.

I am confident with the Python programming language, including the plotting library `matplotlib` and numerical libraries such as `numpy` and `pandas` which I will use to improve the efficiency of my code.

D.4 Evaluation

D.4.1 Success Criteria

For the project to be deemed a success, the following must be successfully completed:

1. The implemented `FastMDP` algorithm produces valid pursuer and evader behaviour.
2. The simulation computes agent actions for each timestep within a reasonable time frame, ensuring the algorithm is computationally efficient.
3. Any explored extensions, such as additional agents or constraints, produce valid and consistent results.

D.4.2 Evaluation Plan

The evaluation will focus on the following:

- **Trajectory validity:** Check distances and angles between agents at each timestep to ensure realistic flight paths.
- **Performance:** Measure CPU time and memory per timestep, and see how this changes with more agents or longer simulations.

D.5 Timetable and Milestones

Weeks 1 to 2 (13 Oct 25 - 26 Oct 25)

Submit Proposal.

Initialise a Git repository for the project and set up a basic python project structure.

Install any dependencies or libraries required and familiarise myself with the basic syntax and usage.

Continue to research Markov Decision Processes and the `FastMDP` algorithm and consider how it can be adjusted for my project.

Weeks 3 to 4 (27 Oct 25 - 9 Nov 25)

Implement a timestep system to update the positions of the agents in the simulation and create a movement path.

Connect the timestep system to a basic visualisation using `matplotlib`, initially testing with pre-defined paths.

Milestone: Generate static plots capable of displaying the path of an agent in a 3D environment.

Weeks 5 to 6 (10 Nov 25 - 23 Nov 25)

Create agent classes to represent the pursuers and evaders with plottable positions.

Begin to consider the reward functions used in the MDP for the pursuers and evaders considering how they will affect movements.

Weeks 7 to 8 (24 Nov 25 - 7 Dec 25)

Begin to implement kinematic constraints and calculations for agent positions.

Implement the reward functions for the pursuers and evaders based on agent positions and velocities.

Milestone: Agents capable of independent movement based on kinematic inputs and constraints and path correctly plotted.

Weeks 9 to 10 (8 Dec 25 - 21 Dec 25)

Finish implementation of kinematic constraints and calculations for agent positions.

Begin implementing the FastMDP algorithm using the kinematic properties of agents, and apply positional updates to the agents at each timestep.

Weeks 11 to 12 (22 Dec 25 - 4 Jan 26)

Slow implementation progress for Christmas and New Year holidays.

Take slack time to ensure all milestones have been met and that the project is on track.

Weeks 13 to 14 (5 Jan 26 - 18 Jan 26)

Continue implementing the FastMDP algorithm.

Start to consider how the MDP or FastMDP algorithm could be adjusted to introduce additional constraints or agents.

Milestone: Kinematic models of agents complete and accurate.

Weeks 15 to 16 (19 Jan 26 - 1 Feb 26)

Finish implementing FastMDP, ensuring that it uses the correct kinematic inputs and applies the correct positional updates to the agents.

Begin writing Implementation draft in dissertation.

Write progress report draft.

Weeks 17 to 18 (2 Feb 26 - 15 Feb 26)

Begin evaluating the project core against the success criteria and tests.

Create progress report presentation.

Finish writing Implementation draft in dissertation.

Milestone: Submit progress report.

Weeks 19 to 20 (16 Feb 26 - 1 Mar 26)

Take slack time to ensure all milestones have been met and that the project is on track.

Begin writing Evaluation draft in dissertation.

Weeks 21 to 22 (2 Mar 26 - 15 Mar 26)

Finish writing Evaluation draft in dissertation.

Begin exploring extensions, focussing on introducing additional constraints to the simulation and decision process.

Milestone: Implementation and Evaluation drafts complete.

Weeks 23 to 24 (16 Mar 26 - 29 Mar 26)

Continue exploring extensions, focussing on introducing additional agents to the simulation and promoting collaborative strategies.

Write Introduction draft in dissertation.

Weeks 25 to 26 (30 Mar 26 - 12 Apr 26)

Finish exploring extensions, and extend Implementation and Evaluation sections in dissertation to discuss explored extensions.

Write Conclusion draft in dissertation.

Milestone: Have a final code repository with the project core implementation and explored extensions complete.

Weeks 27 to 28 (13 Apr 26 - 26 Apr 26)

Combine written dissertation sections into a first full draft, ensuring correct formatting and presentation.

Submit draft to supervisors for feedback and make improvements.

Weeks 29 - 30 (27 Apr 26 - 10 May 26)

Continue to submit dissertation drafts to supervisors for feedback and make improvements.

Ensure that the codebase and the dissertation document are presentable and ready for submission.

Week 31 (11 May 26 - 15 May 26)

Submit final dissertation within deadline.

Milestone: Dissertation and code submitted.

D.6 Resource Declaration

I will be using my personal laptop (Macbook Pro 2021 — 16GB RAM, Apple M1 Pro) as my primary machine for software development. As a backup, I will use my personal desktop computer (32GB RAM, Intel 11th Gen Core i9) or a remote server provided by SRCF (student run computing facility). I will continuously backup my code and dissertation with Git version control and push to a remote GitHub repository.

References

[1] Jason Bertram and Jianghai Wei. "Scalable Online Planning for Multi-Agent MDPs". In: *arXiv preprint* (2019). arXiv: 1909.04171.